Лабораторная работа № 7. Коллективные операции в MPI

1. Описание работы

На языке C вычисляется сумма элементов массива $a_i = i + 1$, $i = 0, \ldots, N - 1$. Для сумм используется тип long long (MPI_LONG_LONG_INT), ожидаемое значение - $N(N + 1)/2$. Процесс 0 получает N из аргумента командной строки и передаёт его всем процессам с помощью MPI_Bcast, затем формирует массив, а MPI_Scatter раздаёт каждому процессу N/P элементов (если N не делится на P, вместо него используется MPI_Scatterv с частями разного размера). Каждый процесс считает локальную сумму. MPI_Gather собирает локальные суммы на процессе 0, который складывает их циклом; MPI_Reduce с операцией MPI_SUM даёт ту же сумму процессу 0 одним вызовом; MPI_Allreduce даёт сумму всем процессам. При $N \leq 0$ программа завершает все процессы через MPI_Finalize без зависания. Время измерялось MPI_Wtime и усреднялось по 20 запускам. Компиляция: mpicc -O2 -Wall -Wextra mpi_array_sum.c -o lab7; запуск: mpiexec -n 4 ./lab7 1000000; для сравнения Gather и Reduce вторым аргументом задаётся число повторений: mpiexec -n 4 ./lab7 1000 100000. Эксперименты проводились на процессоре Intel Core i5-12450H (4 производительных и 4 энергоэффективных ядра, 12 логических потоков, кеш L1 данных 48 КБ и L2 1,25 МБ на производительное ядро, общий L3 12 МБ) в WSL Ubuntu 24.04 (доступно 6 ядер) с Open MPI 4.1.6; при P = 8 использовался ключ --oversubscribe.

2. Результаты экспериментов

Таблица 1. Проверка корректности

Число процессов P	Размер массива N	Ожидаемая сумма	Результат Gather	Результат Reduce	Результат Allreduce	Корректно?
2	100	5050	5050	5050	5050	да
4	100	5050	5050	5050	5050	да
4	10000	50005000	50005000	50005000	50005000	да
8	1000000	500000500000	500000500000	500000500000	500000500000	да

Таблица 2. Влияние размера массива (P = 4, 20 запусков)

Число процессов P	Размер массива N	Среднее время, с	Результат корректен?
4	10000	0.007653	да
4	100000	0.003638	да
4	1000000	0.010909	да
4	10000000	0.068712	да

Таблица 3. Влияние числа процессов (N = 1000000, 20 запусков)

Число процессов P	Среднее время, с	S_P	E_P
1	0.006181	1.00	1.00
2	0.006593	0.94	0.47
4	0.011835	0.52	0.13
8	0.015989	0.39	0.05

Таблица 4. Сравнение Gather и Reduce (100000 повторений, среднее по 5 запускам)

Число процессов P	Способ агрегирования	Число повторений	Среднее время операции, с	Результат корректен?
2	Gather + сумма на процессе 0	100000	0.000000085	да
2	Reduce	100000	0.000000132	да
4	Gather + сумма на процессе 0	100000	0.000000504	да
4	Reduce	100000	0.000000359	да
8	Gather + сумма на процессе 0	100000	0.000000978	да
8	Reduce	100000	0.000000626	да

3. Вывод

Во всех запусках результаты Gather, Reduce и Allreduce совпали с ожидаемой суммой $N(N + 1)/2$. При увеличении размера массива время растёт: для $N = 10^5$, 10^6 и 10^7 оно равно 0.0036 с, 0.0109 с и 0.0687 с. Ускорения при увеличении числа процессов получить не удалось: на 2, 4 и 8 процессах программа работала не быстрее, чем на одном. Основное время, по-видимому, уходит не на сложение, а на то, что процесс 0 сам создаёт весь массив и рассылает его остальным. Эта часть не ускоряется с ростом P, а само сложение N/P чисел занимает совсем немного времени. Кроме того, с ростом P увеличиваются затраты на обмен данными между процессами.

Операции отличаются тем, что Gather просто собирает значения со всех процессов в массив на процессе 0, и сумму потом приходится считать самому циклом. Reduce сразу складывает значения через MPI_SUM и отдаёт один результат процессу 0. Allreduce делает то же самое, но результат получают все процессы, поэтому он удобен, когда сумма нужна каждому (иначе пришлось бы делать Reduce, а потом Bcast). В измерениях при P = 4 и 8 Reduce оказался быстрее Gather с суммой примерно в 1.4 и 1.6 раза, а при P = 2, наоборот, быстрее был Gather.